

PROYECTO: Simulador de Supermercado Multihilo

Un supermercado desea analizar el comportamiento de su sistema de atención al cliente con varias cajas abiertas de forma simultánea. Para ello, se plantea desarrollar una **simulación concurrente** en Java que represente la llegada de clientes, su espera en cola y la atención por parte de las cajas, utilizando **programación multihilo**.

El objetivo es reproducir un escenario realista en el que varios procesos se ejecutan en paralelo, gestionando correctamente los **recursos compartidos**, evitando **condiciones de carrera** y obteniendo **métricas fiables** sobre el rendimiento del sistema.

El sistema simula el funcionamiento de un supermercado con:

- **Clientes** que llegan al supermercado a intervalos variables.
- **Cajas** que atienden clientes de forma concurrente.
- Una **cola compartida** donde esperan los clientes.
- Un **generador de clientes** que produce nuevos clientes durante la simulación.
- Un **módulo de estadísticas** que recoge datos del funcionamiento.

Cada **cliente** será atendido por una sola caja y abandonará el sistema una vez finalizada la atención.

Requisitos funcionales**RF01 Clientes**

Cada cliente deberá tener, como mínimo:

- Identificador único.
- Número de artículos a comprar.
- Instante de llegada al supermercado.
- Instante de inicio de atención.
- Instante de salida del sistema.

El número de artículos será aleatorio dentro de un rango configurable.

RF02 Generador de clientes

- Funcionará como **productor**.
- Creará clientes a intervalos aleatorios (configurables).
- Insertará los clientes en la cola compartida.
- Se detendrá cuando finalice el tiempo total de simulación.

RF03 Cola de clientes

- Será un **recurso compartido** entre generador y cajas.
- Tendrá una **capacidad máxima limitada**.
- Si la cola está llena: El cliente podrá **abandonar el supermercado** (debe registrarse).
- Se recomienda el uso de BlockingQueue.

RF04 Cajas

- Cada caja funcionará como un **consumidor**.
- Atenderá clientes de la cola mientras el sistema esté activo.
- El tiempo de atención dependerá del número de artículos del cliente.
- Deberá registrar tiempos de atención y número de clientes atendidos.
- Las cajas se ejecutarán en paralelo.

RF05 Control de la simulación

- La simulación tendrá una **duración total configurable** o finalizará tras atender un número máximo de clientes.
- El sistema debe cerrarse de forma **ordenada**, sin hilos bloqueados.
- Todas las cajas deben finalizar correctamente antes de mostrar el informe final.

El sistema deberá calcular y mostrar, al menos:

- Número total de clientes generados.
- Número de clientes atendidos.
- Número de clientes que abandonan por cola llena.
- Tiempo medio de espera.
- Tiempo medio de atención.
- Tiempo total de simulación.
- Número de clientes atendidos por cada caja.

Al finalizar la simulación, se mostrará un **informe final por consola** con las métricas calculadas, de forma clara y estructurada.

Ejemplo orientativo:

--- INFORME FINAL ---

Clientes generados: 320

Clientes atendidos: 290

Clientes abandonados: 30

Tiempo medio de espera: 1.25 s

Tiempo medio de atención: 2.10 s

Caja 1 -> 98 clientes

Caja 2 -> 102 clientes

Caja 3 -> 90 clientes

RÚBRICA –Simulador de Supermercado**1. Diseño general y estructura del proyecto (1.5 puntos)**

Nivel	Descripción
Excelente (1.5)	Estructura de paquetes clara y coherente. Separación correcta entre modelo, lógica, concurrencia y métricas. Código modular y bien organizado.
Adecuado (1.0)	Estructura correcta con pequeñas inconsistencias o clases sobrecargadas.
Básico (0.5)	Estructura confusa o poco modular.
Insuficiente (0.0)	Código desorganizado o en una sola clase.

2. Implementación del modelo Productor–Consumidor (2.0 puntos)

Nivel	Descripción
Excelente (2.0)	Uso correcto de BlockingQueue. Productor y consumidores bien implementados y sincronizados.
Adecuado (1.5)	Modelo correcto pero con decisiones mejorables (gestión de parada, límites).
Básico (1.0)	Productor–consumidor funcional pero con errores menores o uso parcial.
Insuficiente (0.0)	No se respeta el modelo o hay fallos graves de concurrencia.

3. Gestión de hilos y ejecución concurrente (2.0 puntos)

Nivel	Descripción
Excelente (2.0)	Uso correcto de ExecutorService. Creación, ejecución y cierre ordenado de los hilos.
Adecuado (1.5)	Uso correcto pero con cierre poco limpio o control mejorable.
Básico (1.0)	Uso directo de Thread o gestión incompleta del ciclo de vida.
Insuficiente (0.0)	Hilos mal gestionados, bloqueos o finalización incorrecta.

4. Sincronización y seguridad en concurrencia (2.0 puntos)

Nivel	Descripción
Excelente (2.0)	No existen condiciones de carrera. Uso adecuado de clases atómicas y colecciones concurrentes.
Adecuado (1.5)	Sincronización correcta con algún punto mejorable.
Básico (1.0)	Existen riesgos potenciales de concurrencia sin manifestarse claramente.
Insuficiente (0.0)	Condiciones de carrera evidentes o datos inconsistentes.

5. Métricas y estadísticas del sistema (1.5 puntos)

Nivel	Descripción
Excelente (1.5)	Todas las métricas solicitadas calculadas correctamente y presentadas con claridad.
Adecuado (1.0)	Métricas principales correctas, pero faltan algunas secundarias o el formato es mejorable.
Básico (0.5)	Métricas incompletas o parcialmente incorrectas.
Insuficiente (0.0)	No se calculan métricas o son incorrectas.

6. Informe final y salida por consola (1.0 punto)

Nivel	Descripción
Excelente (1.0)	Informe claro, ordenado y fácilmente interpretable.
Adecuado (0.75)	Informe correcto con formato mejorable.
Básico (0.5)	Informe confuso o poco claro.
Insuficiente (0.0)	No hay informe final.

7. Calidad del código y buenas prácticas (1.0 punto)

Nivel	Descripción
Excelente (1.0)	Código legible, comentado, con nombres adecuados y sin redundancias.
Adecuado (0.75)	Código comprensible con pequeños problemas de estilo.
Básico (0.5)	Código funcional pero poco legible o mal documentado.
Insuficiente (0.0)	Código difícil de entender o incorrecto.